Höhere Technische Bundeslehranstalt
und Bundesfachschule
im Hermann Fuchs Bundesschulzentrum

Former Glove

AI Training Gloves

Diplomarbeit

Schulautonomer Schwerpunkt

Coding & AI

ausgeführt im Schuljahr 2025/2026 von:

Daniel Jones, 5AHELS

Kevin Silajdzic, 5AHELS

Betreuer/Betreuerin:

Dipl-Ing. Mario Preishuber

Braunau, am 21. Jänner 2026

Eidesstattliche Erklärung

Wir erklären an Eides statt, dass wir die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als angegebene Quellen und Hilfsmittel nicht direkt benutzt und die benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht haben.

Braunau, 07.04.2026	……..……..
Ort und Datum	Vorname Nachname

Braunau, 07.04.2026	……..……..
Ort und Datum	Vorname Nachname

Abstract

Effective strength training relies heavily on correct technique, yet many athletes perform exercises with poor form, increasing the risk of injury and reducing training efficiency. While professional coaching provides a solution, it is often cost-prohibitive or inaccessible for the average gym-goer.

To address this problem, we developed Former Glove, a smart wearable system that acts as a digital coach. The project consists of sensor-equipped gloves that track hand movements during weightlifting. Using intelligent software, the system analyzes the exercise in real-time to detect mistakes and count repetitions automatically. This feedback is wirelessly transmitted to a mobile app, allowing the user to see exactly how well they performed each set.

By offering immediate, automated guidance, Former Glove provides the benefits of professional coaching without the high cost, helping users to train more safely and effectively.

Kurzfassung

Effektives Krafttraining basiert maßgeblich auf korrekter Technik. Viele Athleten führen Übungen jedoch mit fehlerhafter Form aus, was das Verletzungsrisiko erhöht und die Trainingseffizienz mindert. Professionelles Coaching bietet zwar eine Lösung, ist aber für den durchschnittlichen Fitnessstudiobesucher oft zu teuer oder nicht zugänglich.

Um dieses Problem zu lösen, haben wir „Former Glove“ entwickelt, ein intelligentes tragbares System, das als digitaler Trainer fungiert. Das Projekt besteht aus mit Sensoren ausgestatteten Handschuhen, die die Handbewegungen beim Gewichtheben erfassen. Mithilfe intelligenter Software analysiert das System die Übung in Echtzeit, erkennt Fehler und zählt die Wiederholungen automatisch. Dieses Feedback wird drahtlos an eine mobile App übertragen, sodass der Nutzer genau sehen kann, wie gut er jeden Satz ausgeführt hat.

Durch die sofortige, automatisierte Anleitung bietet „Former Glove“ die Vorteile professionellen Coachings ohne die hohen Kosten und hilft Nutzern, sicherer und effektiver zu trainieren.

Contents

Abstract **3**

Kurzfassung **4**

1 Introduction **7**

1.1 Problem Setting 7
1.2 Problem Statement 8
1.3 Solution 8
1.4 Structure of this Thesis 9

2 System Design **11**

2.1 Overview 11
2.2 Hardware Design 12
2.3 Smartphone App Design 12
2.3.1 Application Structure 13

3 Technological Foundation (Kevin) **18**

3.1 Arduino Nicla Sense ME 18
3.2 Edge Impulse 19
3.3 Smartphone App 19
3.3.1 Flutter Framework 19
3.3.2 The Dart programming language 19
3.3.3 Android Studio 20
3.4 Communication Protocols 20
3.4.1 Bluetooth Low Energy 20
3.4.2 Data Transmission Mechanism: Notifications 22

4 Implementation **23**

4.1 Glove Firmware Logic 23
4.1.1 Data Collection 24
4.1.2 App Test 29
4.1.3 Final Code 31
4.2 3D Print 33
4.2.1 Case Measurements 33

4.2.2 Mounting Rail 34
4.2.3 Slicing & Printing 35
4.3 AI Model Training 36
4.3.1 Form Detection 37
4.3.2 Rep Detection 40
4.4 App Development 42
4.4.1 Early Prototype BLE Data Transfer Test 42
4.4.1.1 Monolithic Architecture 43
4.4.1.2 Raw Data Parsing 43
4.4.1.3 UI Rendering and Limitations 44
4.4.2 Architecture and State Management Approach 45
4.4.3 Ensuring Continuous BLE Connectivity via Global State 45
4.4.4 Two-Way BLE Communication Module 46
4.4.5 Real-Time UI activity (Active Workout Screen) 48
4.4.6 Data Serialization and Local Storage 49
5 User Case 51
6 Evaluation 52
7 Project Management 53
7.1 Planning 53
7.2 Reflection 54
7.3 Timesheet 56
7.3.1 Daniel Jones 56
7.3.2 Kevin Silajdzic 59
8 Future Work 61
9 Related Work 62
10 Conclusion 63
Acknowledgements 64
List of Figures 65

1 Introduction

Weight training has become one of the most popular forms of exercise worldwide. Whether for aesthetic reasons, rehabilitation, or general health, millions of people lift weights every day. However, simply moving a weight from point A to point B is not enough. The key to success lies in the quality of the movement.

1.1 Problem Setting

Imagine a beginner walking into a gym for the first time and trying to do bicep curls with no prior experience. Instead of using only their arm muscles, they swing their entire upper body to lift the weight. To a trained professional, the mistake is obvious. To a beginner, however, it feels like a success because the weight moved up.

This scenario illustrates a fundamental problem in individual sports: The lack of immediate and objective feedback.

Performing exercises with incorrect form has two major negative consequences. First, it drastically reduces the effectiveness of the training, as the target muscles are not properly engaged. Second, and more importantly, it significantly increases the risk of acute and chronic injuries.

While professional athletes rely on personal trainers to monitor their form, this solution is not scalable for the general public. Personal trainers are expensive, often costing significantly more per hour than the monthly gym membership itself, and they cannot be present for every single workout session a person does. Consequently, a vast number of gym-goers are left to train unassisted, unaware of their mistakes, and prone to injury.

1.2 Problem Statement

The central problem is the absence of an accessible, technological solution that can substitute the help of a professional trainer.

While wearable technology is widely available in the sector of endurance sports (tracking heart rate, steps, or distance), solutions for strength training remain largely underdeveloped. Existing fitness trackers primarily focus on quantitative data (e.g., "How many reps did I do?"), but they fail to capture qualitative data (e.g., "Was the movement performed correctly?").

Furthermore, stationary alternatives such as camera-based systems are often impractical for gym environments due to the lack of mobility and privacy concerns. Consequently, there is currently no mobile, affordable, and privacy-compliant system available that can autonomously analyze biomechanical movement patterns in real-time and provide corrective feedback to the user.

1.3 Solution

To solve this problem, we invented the "Former Glove" system. The core idea is to replace the eyes of a personal trainer with a smart wearable device that is affordable, always available, and objective.

The solution consists of two main components:

1. **The Former Glove:** A specialized fitness glove equipped with sensitive motion sensors. These sensors act as the system's "sense of touch", recording the precise movement patterns of the user’s hands in a three-dimensional space.
2. **The Former App:** A mobile application that acts as the user interface and "digital coach”, using Bluetooth to connect to the gloves.

Unlike simple fitness trackers that only count steps or repetitions, the Former Glove system understands the quality of a movement. Using artificial intelligence, the system analyzes the sensor data in real-time. It determines whether an exercise (like a bicep curl) was performed cleanly or with momentum.

The contribution of this work is a fully functional prototype that connects the physical and digital worlds. It allows a user to define a workout, perform it wearing the gloves, and receive immediate feedback on their smartphone regarding repetition counts and execution quality. This allows the user to correct their form instantly, ensuring a safer and more effective workout without the need for a human instructor.

1.4 Structure of this Thesis

To provide a comprehensive understanding of this project, this thesis is structured in a logical progression, transforming unassisted self-training into a digitally coached fitness experience. The integrated narrative is organized into the following chapters:

Chapter 1 (Introduction) establishes the foundation by contextually defining the problems beginners face during strength training. It presents the unified problem statement, introduces the hybrid glove-and-application solution, and outlines the organizational structure that guides this engineering venture.

Chapter 2 (System Design) details the comprehensive architectural blueprint of the entire ecosystem. It presents unified data flow diagrams, defines the physical hardware design required for the smart gloves, and outlines the foundational frontend design principles used to create an intuitive mobile application experience.

Chapter 3 (Technological Foundation) provides the theoretical groundwork required to understand the project's distinct elements. It merges the hardware perspective, introducing the Arduino Nicla Sense ME and the principles of Edge AI via Edge Impulse, with the software stack, detailing the Flutter framework and the intricate Bluetooth Low Energy communication protocols that unify the gloves and smartphone.

Chapter 4 (Implementation) shows the practical realization of the integrated system. It details how the distinct components, the glove's firmware logic and 3D-printed housing, the training of the specific AI models for form and repetition detection, and the development of the mobile application, were realized. Crucially, this chapter describes the critical evolutionary step from a rudimentary raw-data test version to the final, bandwidth-optimized and state-driven smart system.

Chapter 5 (User Case) serves as the physical proof of concept by demonstrating the system's seamless operation in a real-world workout. By walking through a real training scenario, this chapter shows the glove and app working in symphony: from user setup to real-time AI-based form analysis and automated digital coaching delivery.

Chapter 6 (Evaluation) provides objective data regarding the unified system's effectiveness. It analyzes the accuracy of the glove's form detection and repetition counting, while simultaneously measuring the app's latency and responsiveness to ensure the solution provides meaningful, real-time value to the athlete.

Chapter 7 (Project Management) and **Chapter 8 (Future Work)** document the process and outlook of the team. Chapter 7 includes the planning phases, joint reflections from both authors, and total signed timesheets, while Chapter 8 discusses potential expansions for the combined hardware and software capabilities based on this functional prototype.

Finally, **Chapter 9 (Related Work)** compares the Former Glove solution against existing commercial alternatives, while **Chapter 10 (Conclusion)** summarizes the major achievements of this comprehensive smart fitness ecosystem.

2 System Design

2.1 Overview

The final project includes three components, all of which have to communicate with each other periodically. Firstly, we have the right glove, which only has to send its sensor values to the left glove so they can be evaluated together. The right glove is a server and the left glove connects to it as a client via Bluetooth Low Energy (BLE). The left glove serves both as a BLE client and server, as the mobile app is connected to it over BLE as a client. It is responsible for evaluating the combined sensor time series data using an AI model for form classification, while the right glove handles the repetition recognition AI. While the left glove sends both signals for the end of a rep and the evaluated label, the app only has to communicate whenever the user starts a new set of exercises, including the selected exercise, weight and repetitions.

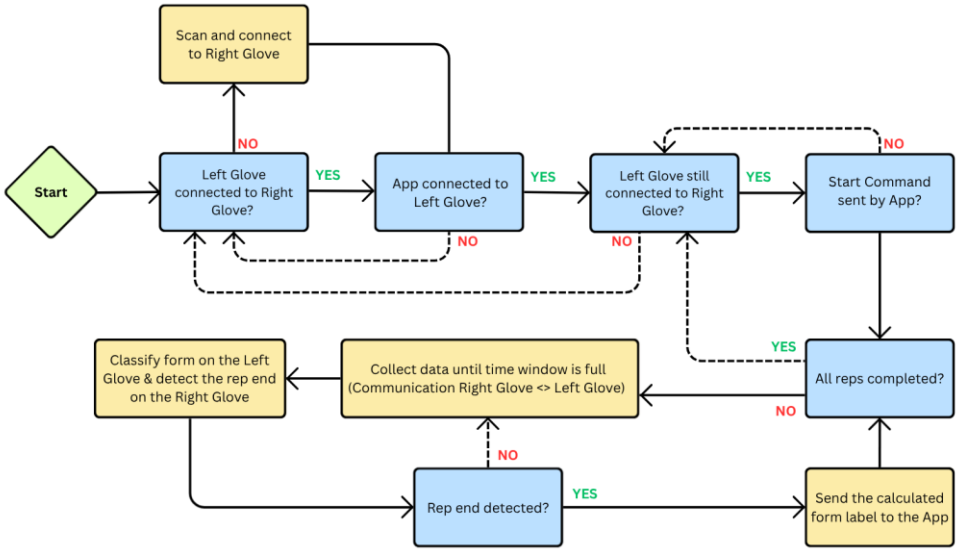


Figure 1: Process

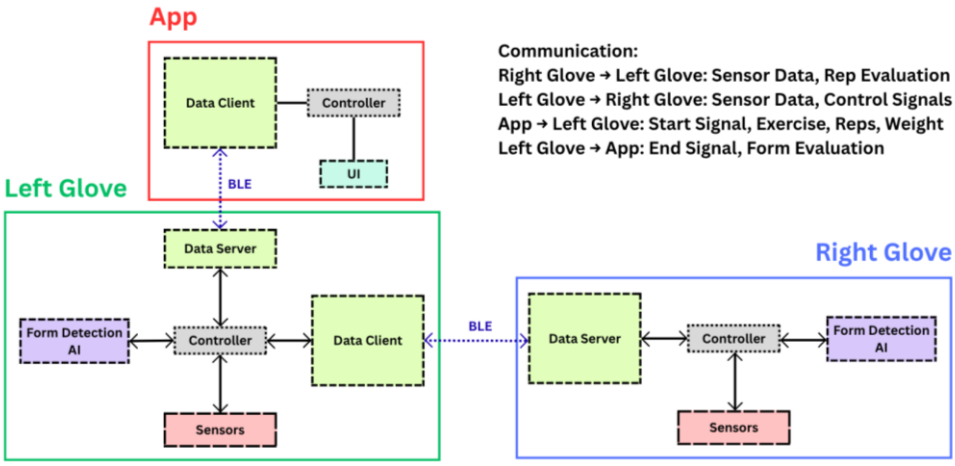


Figure 2: System Architecture

2.2 Hardware Design

The hardware design is a challenge, as training gloves require free movement and too much clutter would hinder the user in exercising. Thus, the best solution that minimizes the size of both the microcontrollers' and battery's housing is modelling a mounting rail. The housings are connected to the counterpart of the rail's shape, which makes it possible to slide them together without the need for any screws or clips. The mounting rail has small holes so it can be sewn to the glove.

For the battery, a smaller model with 250mAh is used on both gloves. To charge the microcontrollers, it can easily be connected to the battery pins of the Nicla Sense ME by connecting a plug directly leading to the battery and a socket on the microcontroller's side. The battery will automatically be charged when the Nicla Sense ME is connected via micro-USB cable.

2.3 Smartphone App Design

The mobile application Former serves as the visual bridge between the data processed by the smart glove and the athlete. The primary design goal is to translate raw data and AI classifications into an intuitive, easy-to-understand digital coaching experience.

Designing a user interface for a fitness environment presents unique challenges. Users are physically active and their primary focus must remain on the exercise, not on their smartphone. Therefore, the User Experience strategy for the Former app is built on three core principles:

1. Minimal Touch Interaction: During an active workout, the user should interact with the smartphone screen as little as possible. The application is designed to be highly automated. Once a training session is configured and started, the progression through sets is handled entirely by the app using the incoming sensor data. This prevents the user from having to manually log repetitions or tap "next" with sweaty hands.

2. High Visual Clarity: Smartphones are frequently placed on the floor, on a bench, or propped against a water bottle during workouts. Consequently, the screen must be readable from one to two meters. The UI shows critical metrics at a large scale and uses high-contrast colors to ensure readability of the live repetition count and AI Feedback.

3. Global State Indication: Because the application relies on a continuous BLE connection to the glove, the user must always be aware of the connection status. Instead of hiding this information in a separate settings menu, a persistent visual connection status indicator is anchored in the top navigation bar across all screens. This provides reassurance that the hardware is actively communicating with the software.

2.3.1 Application Structure

The application follows a simple navigation flow, guiding the user from the launch to the final performance review. The architecture consists of five primary screens, each serving a distinct purpose in the user journey:

1. The Start Screen: After launching the app, the user is presented with a clean Start Screen that establishes the brand identity. It prominently features the product logo and name and a primary start button, providing a clear entry point into the application.

2. The Main Menu: The Main Menu acts as the central point of navigation. At the top of the screen, the global BLE connection status is displayed, showing whether the glove is currently searching, connected, or disconnected. The main body of the screen provides a list of all available exercises that the AI is trained to classify (e.g. Bicep Curls, Lateral Raises, Bench Press). From here, the user can either tap a button to view their past workout history or enter the "Create Set" mode. In this mode, exercises can be selected sequentially to build a custom workout routine.

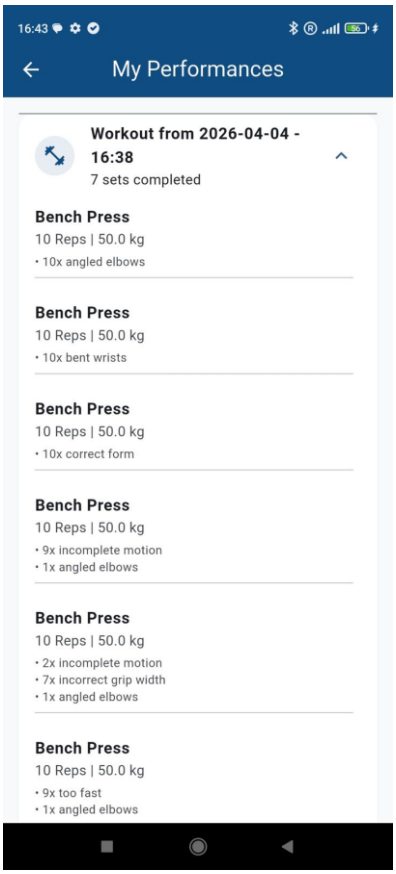


Figure 3: Workout History

3. The Setup Screen: After selecting the desired exercises, the user navigates to the Setup Screen. This interface provides input fields to configure the specific parameters for each chosen exercise. The user defines the number of sets, the target repetitions per set, and the weight being used (in kilograms) for every exercise type. This configuration is essential, as it dictates the underlying logic for the automated tracking phase.

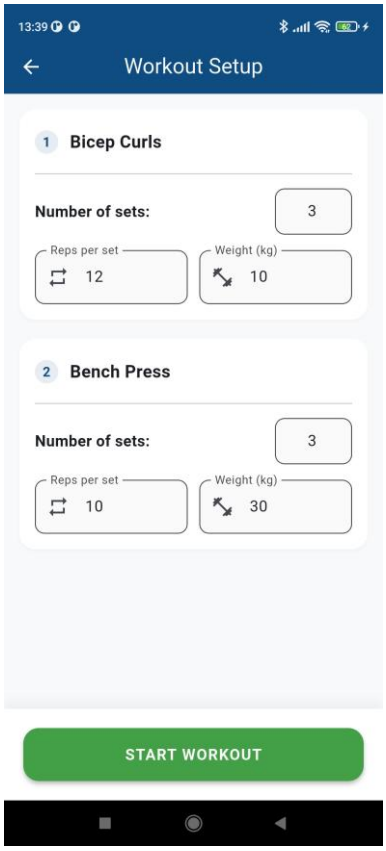


Figure 4: Workout Setup

4. The Active Workout Screen (Real-Time Tracking): This screen prioritizes the minimal touch interaction principle. It features a large, highly visible repetition counter that increments automatically based on real-time data received from the glove. To maintain the flow of the workout, this screen has automated transitions from one exercise to the next when the user reaches the previously configured repetition count.

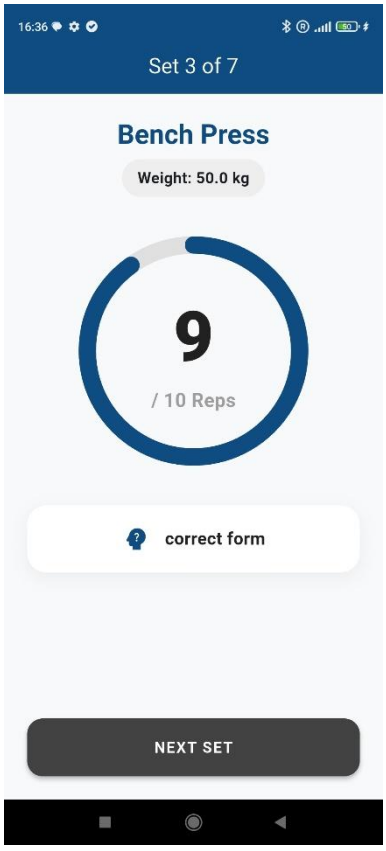


Figure 5: Running Set

5. The Summary Screen: Upon completing the final set of the configured workout, the active tracking stops and the user is directed to the Summary Screen. This screen provides an overview of the completed workout, listing the actual achieved repetitions versus the targets. Most importantly, it presents the qualitative AI-generated feedback in readable text, highlighting execution errors (such as using too much momentum) and providing corrective advice. A final action button allows the user to save the session data and return to the Main Menu.

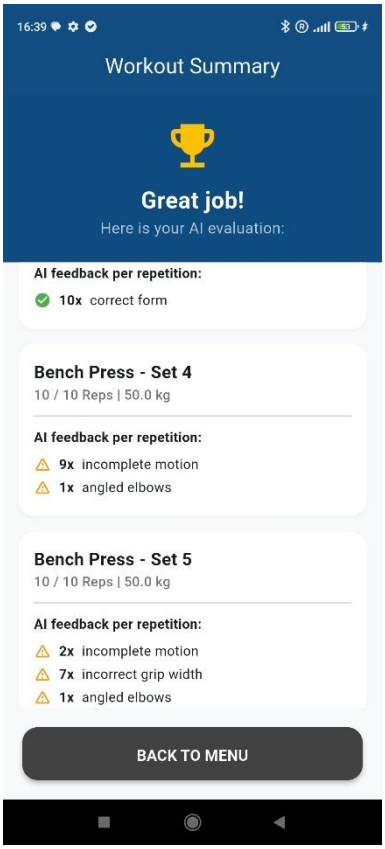


Figure 6: Summary Screen

3 Technological Foundation (Kevin)

The implementation of a smart tracking system like the Former Glove requires the integration of different technologies: embedded hardware, artificial intelligence, mobile software and wireless communication protocols.

This Chapter provides the knowledge needed to understand the design and implementation strategies described later in this thesis. It introduces the individual hardware components used for motion tracking, the software frameworks used to train the artificial intelligence Model and to build the smartphone app and the properties of the Bluetooth Low Energy protocol which enables the wireless communication of data between devices.

3.1 Arduino Nicla Sense ME

The Nicla Sense ME is a microcontroller by Arduino. Its miniscule size is practical for integrating it into smaller devices without creating bulk. It is equipped with various sensors provided by Bosch, including an accelerometer and a gyroscope, which we will be using to track the movement of our gloves. It is also specifically designed for running complex AI-based code, which makes it a perfect fit for this project. Two of them will be used, one for each glove.

(Source: https://docs.arduino.cc, 03-26-2026)



Figure 8: Arduino Nicla Sense ME

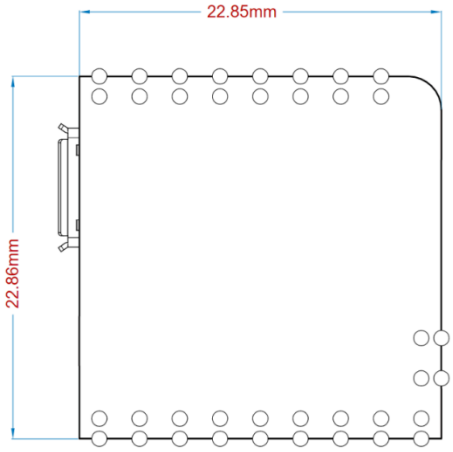


Figure 7: Arduino Nicla Sense ME Mechanical Details

(Sources: https://reichelt.at, https://docs.arduino.cc)

3.2 Edge Impulse

Edge Impulse is a development platform for embedded machine learning. It enables the collection, preprocessing, and labelling of sensor data, as well as the training and optimization of AI models directly for microcontrollers. It will process the movement data collected and help create a library which can be directly run on the microcontroller.

(Source: https://docs.edgeimpulse.com, 03-26-2026)

3.3 Smartphone App

The user interface (UI) and the logic for workout management are hosted on a mobile application. The Former app was developed using the Flutter framework in combination with the Dart programming language and the Android Studio integrated development environment (IDE).

3.3.1 Flutter Framework

Flutter is an open-source UI software development kit (SDK) developed by Google. It allows developers to build applications for mobile, web, and desktop from a single codebase. Unlike traditional development (which would require Kotlin for Android and Swift for iOS), Flutter uses its own rendering engine to draw widgets. This makes sure that the user has a consistent visual appearance across different devices. In Flutter, the entire UI is built by nesting widgets (e.g., buttons, text, layout containers) into a tree structure. This approach is essential for the Former app to dynamically rebuild widgets when new sensor data arrives via Bluetooth.

(Sources: https://flutter.dev/, https://docs.flutter.dev/, 03-25-2026)

3.3.2 The Dart programming language

The Flutter SDK uses the Dart programming language also developed by Google. Dart is optimized for object-oriented programming. Dart is particularly well suited for the development of the app because of two main features:

- Just-in-Time (JIT) Compilation: This enables the "Hot Reload" feature during development, allowing code changes to be reflected instantly in the running app without losing the current state (e.g., a simplified Bluetooth connection).
- Asynchronous Programming: Dart supports asynchronous operations using Future and Stream objects. This is critical for handling Bluetooth Low Energy (BLE) events, where data packets from the glove arrive asynchronously and must be processed without freezing the user interface.

(Source: https://dart.dev/overview, 03-25-2026)

3.3.3 Android Studio

The development environment used is Android Studio, the official IDE for Android development. It provides the necessary tools for code editing and debugging.

Although Android Studio includes an integrated device emulator for virtual testing, this project relies heavily on the Android Debugging Bridge (ADB), which acts as an interface between the development environment and the physical device. Since standard emulators don't support the use of Bluetooth connections, uploading the application to a smartphone was essential for testing the communication with the Former Glove.

(Source: https://developer.android.com/studio/intro, 03-25-2026)

3.4 Communication Protocols

3.4.1 Bluetooth Low Energy

For the wireless data transmission between the wearable hardware and the mobile application, Bluetooth Low Energy (BLE) was selected as the communication standard. BLE is specifically designed for applications requiring low power consumption and sporadic data transfer, making it the industry standard for fitness wearables.

Unlike classic Bluetooth, which is optimized for continuous data streaming (e.g., audio), BLE remains in sleep mode for the majority of the time and only wakes up to transmit short bursts of data. This characteristic is essential for the Former Glove, as it allows the Nicla Sense ME

to operate with a minimal battery consumption while maintaining a connection over the duration of a workout session.

(Source: https://en.wikipedia.org/wiki/Bluetooth_Low_Energy, 03-25-2026)

3.4.1.1 Generic Access Profile (GAP)

The Generic Access Profile (GAP) defines how devices discover each other and establish a connection. In this project, the system architecture assigns specific roles to the hardware components:

- **Peripheral (The Glove):** The Nicla Sense ME acts as the Peripheral. It is a low-power device that "advertises" its presence by broadcasting small data packets containing its name ("LeftGlove") and available services. It waits for a connection request.
- **Central (The Smartphone):** The mobile device running the Flutter application acts as the Central. It has higher processing power and battery capacity. Its role is to scan for advertising packets, initiate the connection, and manage the communication timing.

(Source: https://developer.android.com/develop/connectivity/bluetooth/ble/ble-overview, 03-25-2026)

3.4.1.2 Generic Attribute Profile (GATT)

Once a connection is established via GAP, the **Generic Attribute Profile (GATT)** defines how data is structured and exchanged. GATT organizes data hierarchically into **Services** and **Characteristics**:

1. **Services:** A Service is a logical collection of data and behaviors to accomplish a specific function. For this project, a custom "Training Service" was defined and identified by a unique UUID (Universally Unique Identifier).
2. **Characteristics:** Characteristics are the actual data containers within a Service. They hold values (e.g., the repetition count or the sensor feedback) and properties that define how this value can be accessed (Read, Write, or Notify).
3. **UUIDs:** To ensure that the app communicates with the correct data points, both the Service and its Characteristics are addressed via predefined UUIDs known to both the firmware and the app software.

(Source: https://developer.android.com/develop/connectivity/bluetooth/ble/ble-overview, 03-25-2026)

3.4.2 Data Transmission Mechanism: Notifications

Efficient data transfer is critical for real time feedback. Standard polling (where the app constantly asks "Do you have new data?") is inefficient and increases latency. Instead, this project utilizes the notification mechanism provided by BLE. The smartphone "subscribes" to the specific characteristic of the glove. Whenever the glove's internal logic detects a new event (e.g., a completed repetition), it actively pushes the update to the smartphone. This event-driven architecture ensures that the application interface updates instantly while minimizing the Bluetooth usage of the microcontroller.

(Source: https://developer.android.com/develop/connectivity/bluetooth/ble/transfer-ble-data, 03-25-2026)

4 Implementation

4.1 Glove Firmware Logic

The code for the microcontrollers is programmed in C++ using Visual Studio Code as the editor and PlatformIO as the compiler. The whole structure is managed in a GitHub repository and documented in multiple commits. Multiple main files will be needed at all times as the code varies between the two gloves and creating a unique project for every iteration would be very disorganized. To separate them from each other, every main file has to have its own framework in the initialization file, which all have a similar structure that filters them by name:

```
[env:leftGloveFetchData]
platform = nordicnrf52 # The platform to build for
board = nicla_sense_me # The specific board to build for
framework = arduino # The framework to use
monitor_speed = 115200 # The baud rate for the serial monitor
build_src_filter = +<*.h> +<main-${PIOENV}.cpp> # Searching by Appendix after main
                                                so multiple main files can be used
lib_deps =
    arduino-libraries/ArduinoBLE@^1.4.1
    arduino-libraries/Arduino_BHY2@^1.0.8 # The libraries to include in the project
```

To make use of BLE, the sensors and the trained AI models, there are multiple libraries which have to be included in the corresponding codes:

ArduinoBLE:

The ArduinoBLE libraries allows Arduino boards with Bluetooth Low Energy capability to communicate with other BLE devices such as smartphones, sensors, or computers. It provides simple functions to create BLE peripherals or central devices, define services and characteristics, and send or receive data wirelessly. This library is commonly used with boards like the Arduino Nano 33 BLE or Arduino Nicla Sense ME to build IoT and sensor applications that require low-power wireless communication.

(Source: https://docs.arduino.cc, 03-26-2026)

Arduino_BHY2:

The Arduino_BHY2 library is used to access and manage the sensors integrated in boards like the Arduino Nicla Sense ME. It communicates with the Bosch BHY2 sensor hub, which processes data from sensors such as the accelerometer, gyroscope, magnetometer, and environmental sensors. The library provides functions to initialize sensors, read sensor data, and use advanced features like sensor fusion and event detection. This makes it easier to collect motion and environmental data for applications such as activity tracking, gesture recognition, or IoT monitoring.

(Source: https://docs.arduino.cc, 03-26-2026)

Edge Impulse Arduino Deployment:

Edge Impulse Arduino deployment allows trained machine learning models to run directly on Arduino-compatible microcontrollers. After collecting and training data in Edge Impulse, the model can be exported as an Arduino library. This library can then be integrated into an Arduino sketch to perform real-time inference on sensor data, such as accelerometer signals from devices like the Arduino Nicla Sense ME. This enables applications such as motion recognition, anomaly detection, or activity classification directly on the device without requiring a constant internet connection.

(Source: https://docs.edgeimpulse.com, 03-26-2026)

4.1.1 Data Collection

The original data has to be collected manually as repetitions can't be detected until a complete data set is created. This function is solely used by the developers and isn't accessible to the users.

As the two gloves aren't physically connected, the sensor values are sent via BLE so both can be accessed by the left glove. Tables fit for the data set are then created out of the movement values, label and timestamp. To create said tables an additional connection between the left glove and the Serial Monitor is created via a COM-Port.

Firstly, the right glove has to advertise itself:

```
BLE.begin();
BLE.setLocalName("RightGlove");
BLE.setAdvertisedService(sensorValues);
sensorValues.addCharacteristic(movementChar);
BLE.addService(sensorValues);
BLE.advertise();
```

The sensor data is collected periodically and written on the characteristic so the left Glove can access it. The easiest way to do it is to copy the values into the buffer memory, as you can send multiple variables as one which are then again separated by the receiver.

```
void sendSensorData(){
  // Retrieve sensor values from the right glove and send to the left glove via BLE

  float ax = accel.x(), ay = accel.y(), az = accel.z();
  float gx = gyro.x(), gy = gyro.y(), gz = gyro.z();

  byte buffer[24];
  memcpy(&buffer[0], &ax, 4);
  memcpy(&buffer[4], &ay, 4);
  memcpy(&buffer[8], &az, 4);
  memcpy(&buffer[12], &gx, 4);
  memcpy(&buffer[16], &gy, 4);
  memcpy(&buffer[20], &gz, 4);

  movementChar.writeValue(buffer, 24, true);
}
```

```
float* handleRightValues() {
// Retrieve sensor values from the right glove from the characteristic and convert
   into float array

  byte buffer[24];
  if (!movementChar.readValue(buffer, 24)) { // Reads from movementChar if available
    Serial.println("Failed to read from RightGlove.");
    return nullptr;
  }

  float ax, ay, az, gx, gy, gz;
  memcpy(&ax, &buffer[0], 4);
  memcpy(&ay, &buffer[4], 4);
  memcpy(&az, &buffer[8], 4);
  memcpy(&gx, &buffer[12], 4);
  memcpy(&gy, &buffer[16], 4);
  memcpy(&gz, &buffer[20], 4);

  static float rightValues[6]; // Number of sensor axes
  rightValues[0] = ax;
  rightValues[1] = ay;
  rightValues[2] = az;
  rightValues[3] = gx;
  rightValues[4] = gy;
  rightValues[5] = gz;

  return rightValues;
}
```

In the code of the left glove, the connection to the right glove is handled by searching for the device by name:

```
void scanForRightHand(){
// Checking if the right glove appears in the BLE scan results and connecting to
   it
  Serial.println("Scanning for RightGlove...");
  while(true){ // It endlessly scans until it finds the right glove, as nothing else
                  should run without this connection
    peripheral = BLE.available();
    if(peripheral) {
      if (peripheral.localName() == "RightGlove") {
        Serial.println("RightGlove found!");
        BLE.stopScan();
        break;
      }
    }
     delay(10); // small delay to avoid 100% CPU usage while scanning -> longer
                   battery life
  }
}
```

The left glove has access to its own sensors and the data from the right glove can simply be read from the characteristic. To save the data into tables fit for the data set, there is also a function to send the whole data to the Serial Monitor.

```
void sendToPC(float* leftValues, float* rightValues) {
  if (!leftValues || !rightValues) return;
  // Go through all sensor axes and output them via Serial

  for (int i = 0; i < 6; i++) {
    Serial.print(leftValues[i], 2);
  }
  for (int i = 0; i < 6; i++) {
    Serial.print(rightValues[i], 2);
  }
}
```

Two people are needed to create the movement data set, one controls the recording process and the other completes the exercises. In order to actually collect the movement data, a python script was created which allows the user to select the currently recorded exercise, weight and label. By pressing the enter button, a recording is started and ended for each repetition. While recording, the table is updated at a rate of 10 Hz. Following values are present in each row:

- 12 movement values
  - 6 per glove
  - 3 accelerometer axes, 3 gyroscope axes
- 6 derivatives
  - 3 per glove
  - difference between accelerometer values from frame to frame
- 2 vector magnitude values
  - 1 per glove
  - combines the accelerometer vectors into one singular vector
  - $acc_{\{norm\}} = \sqrt{\{x^2 + y^2 + z^2\}}$
- 3 difference values
  - The difference between the x, y and z position of the two gloves
- Weight
- Time since rep start
  - increments of approx. 100 ms
- Label
  - Stays the same for the whole rep

Here is an example of the labels available for an exercise, in this context a bench press.

```
LABELS = [
    "correct form",
    "bent wrists",
    "angled elbows",
    "bar not touching chest",
    "incomplete motion",
    "using momentum",
    "incorrect grip width",
    "too fast"
]
```

Another label that's important to determine is whether the current rep is in about to end or not, so that it will be possible to recognise new repetitions automatically. To prepare that data set, the already created one only has to be slightly altered. Another python script is used to split every table into two separate tables. The first 60% are labelled as "RUNNING", the last 40% as "END". This way an AI model can later be created to separate the two phases.

4.1.2 App Test

To confirm the current approach of using the left glove as both a client and a server is working to guarantee correct communication between the gloves and the app later on, a simple connection test is of great importance.

The connection to the right glove is built as usual. Additionally, the left glove is advertised as a BLE server. In the loop function, the Nicla Sense ME waits until the phone connects to the microcontroller in the App via Bluetooth and then starts sending the sensor values over a characteristic. Once the App isn't connected anymore, the left glove readvertises and breaks the loop as nothing should work without the connection in the final code.

```
void loop() {
    BLEDevice central = BLE.central();

    // If a device connects (e.g., app)
    if (central) {
        Serial.print("Connected to central device: ");
        Serial.println(central.address());

        while (central.connected()) {
            float rax = accel.x(); float ray = accel.y();
            float raz = accel.z(); float rgx = gyro.x();
            float rgy = gyro.y(); float rgz = gyro.z();

            // Convert data into byte array (e.g., 4 bytes per value)
            byte data[24];
            memcpy(data, &rax, 4);
            memcpy(data + 4, &ray, 4);
            memcpy(data + 8, &raz, 4);
            memcpy(data + 12, &rgx, 4);
            memcpy(data + 16, &rgy, 4);
            memcpy(data + 20, &rgz, 4);

            // Send data to app via BLE
            movementChar.writeValue(data, sizeof(data));
            delay(50); // 20 Hz send interval
        }

        // Connection disconnected
        BLE.advertise(); // Readvertising as LeftGlove
    }
}
```

In the app, these values were then shown and updated every 50ms. These values fit the corresponding hand movements, e.g. the x-value on the accelerometer increasing drastically when punching forward. This proved that the method worked as planned and the connection wouldn't be an issue going forward.

4.1.3 Final Code

All of the code is related to the left glove if not explicitly stated otherwise. The final code can be separated into four stages:

Setup:

The three libraries used and the firmware are included at the top. For the left glove the form detection model is used, the right glove uses the rep detection. It is important that for the Nicla_System library, multiple defined values have to be undefined as they conflict with values defined in the Edge Impulse library under the same name. This doesn't delete any necessary defines, otherwise they would simply have to be renamed.

Multiple global variables have to be created: The amount of parameters for the AI model as well as values that are necessary to fill the arrays which are evaluated in the classifier functions. Most of these values can be taken by the Edge Impulse library.

After setting up the sensors, three BLE characteristics are needed. One to communicate with the App, one to exchange sensor values between gloves and one for the left glove to send the right glove a single integer which tells it what to do.

Connection Handling:

Before anything else can happen in the loop function, the left glove searches for the right glove until it connects. After that, it checks if the App is also connected and keeps running the rest of the code while the latter connection persists. If at any point the right glove disconnects, the functionality immediately stops and returns to the start of the loop to connect again. This is important because without the glove-to-glove connection the program can't work correctly and continuing to run the evaluation is futile and would ruin the recorded data in the app.

Communication Left Glove <> App:

Before starting the rep evaluation, the left glove has to wait until the start signal at the start of a training set. It sends a starting flag and three values for the selected exercise, weight and the amount of reps. After every evaluated rep, the Nicla Sense ME sends the predicted label to the app. Once all reps are completing, the starting flag is set to false and it once again waits for the app to start a new set.

Evaluation:

At the start of each rep, both the array that holds the current features of the rep detection AI and the one that sums up the total evaluation of each label are set to zero so the old data is overwritten. A signal is sent to the right glove to ensure the other AI's features are overwritten as well.

The sensor values are repeatedly refreshed. Every 100ms, both gloves' values are collected and put into both feature arrays.

```
void fillFeatures_bp_form(float* leftValues, float* rightValues, int index) {
    int start_pos = index * AXES_PER_SAMPLE;
    // Calculate start position in the feature buffer for this sample

    for (size_t i = 0; i < 6; i++) {
        // Loop through the 6 sensor values (acc + gyro)

        features_bp_form[start_pos + i] = leftValues[i];
        // Store left glove sensor values

        features_bp_form[start_pos + i + 6] = rightValues[i];
        // Store right glove sensor values directly after the left values
    }
}
```

Once a window is full, the label gets evaluated and the feature is shifted to the right depending on the window increase. In case of the form detection, the percenages of each value gets added to it's corresponding index in the array. For the rep detection, the label evaluated on the right glove is sent to the left glove and the counters are updated. As the samples are split into 60% running and 40% end, once the end labels make up at least 30% of the total amount of evaluations a rep ends. The running counter also has to be at least two to avoid a repetition immediately ending.

```
// update counters for end-of-rep voting logic
if (!end) { // evaluation sent from the right glove
    total_running++;
} else {
    if (total_running >= 1) { // Ensure the AI detected a new repetition started
        total_end++;
    }

    if (total_running >= 2 &&
       (float)total_end / (float)(total_running + total_end) > 0.3) {
       end_of_rep = true;
    }
}
```

Once the end of the rep was detected, the label evaluates one last time before sending the best performing label to the app.

4.2 3D Print

Autodesk Fusion is used to create the models for 3D printing. For the final version of the prototype Bambu Studio was used to slice the model, every other iteration used PrusaSlicer.

4.2.1 Case Measurements

It is important to keep the housing measurements close to the microcontroller size so it can easily fit on the glove. The Nicla Sense ME is 22.85mm long on both sides, and the walls of-the housing are each 4.25mm wide. Two rectangular holes where added to allow access to the charging port and the battery connectors, the charging port having a thinner wall so that the plug can fit through. There is an M3 screw hole for every corner so a cover could be screwed on later.

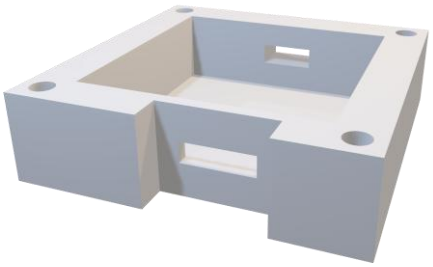


Figure 9: Nicla Sense ME Housing v1

The same procedure was executed for the battery housing, which holds a size of 30.35x16.75mm^2. This time, only one rectangular hole to access the battery connectors were added.

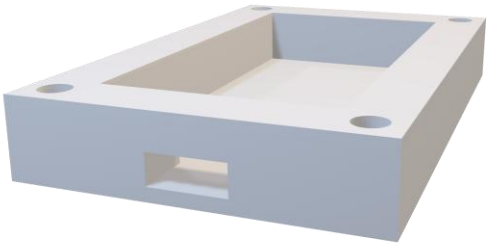


Figure 10: Battery Housing v1

While the size of this version is already perfectly adjusted, using screws to close the covers is suboptimal as the metal wouldn't feel good when using the gloves for training.

4.2.2 Mounting Rail

To mount the housings to the glove without any screws, a T-shaped mounting rail was developed. On each side of the base, there are 12 holes with a diameter of 1mm, through which it is possible to sew the rail to the glove.

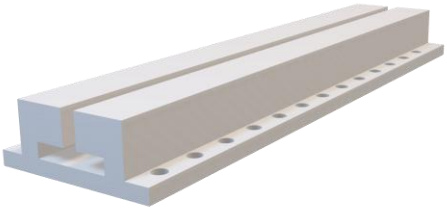


Figure 11: Mounting Rail

For both the microcontroller and battery housings, the counterpart of the mounting rail's T-Shape was added to the top. On each side of the rail, there is a small space so that the rails can easily be slid together and stay in place without the need for any further support. Additionally, the Nicla Sense Me no longer needs the holes to access the ports, as the whole top is now open and everything can easily be reached.

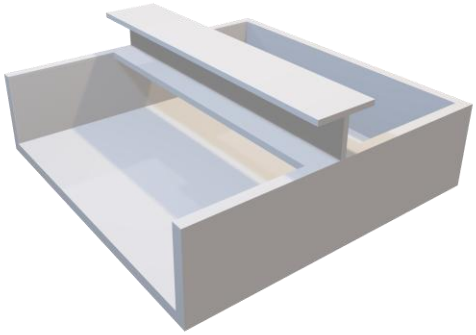


Figure 13: Nicla Sense ME Housing v2

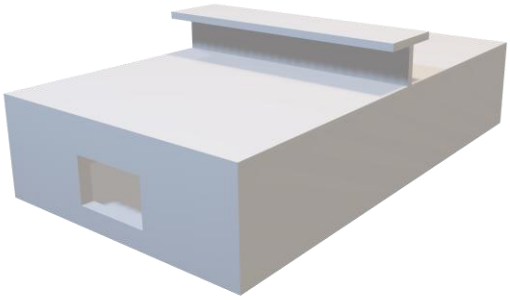


Figure 12: Battery Housing v2

4.2.3 Slicing & Printing

While printing, "tree" supports at an angle of 30 degrees were used. The only way the models print correctly on the BambuLab A1 was to print them at a 90-degree angle, otherwise the filament collapses under its own weight.

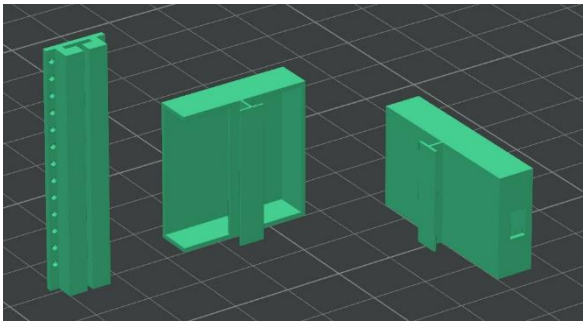


Figure 14: Housing sliced in Bambu Studio

After connecting the plug and socket connectors to the microcontroller and battery, the prototype used for data collection and AI testing is completed.



Figure 15: Prototype

4.3 AI Model Training

In Edge Impulse, two AI models are created: One detects if you are performing the correct form and which mistakes you are making, the other when a repetition ends and a new one starts. These “Impulses” are then deployed into Arduino libraries which can be used on the Nicla Sense ME. Originally, it was planned to run both AIs on one glove, however these libraries all use the same names for their global variables and functions. These conflicting variables are impossible to separate within the same project, so the left glove handles the form detection while the right glove handles the rep detection.

4.3.1 Form Detection

The self-made data set can be easily be inserted into the project by creating a CSV Wizard which handles the uploaded files. The movement is time series data, which is tracked by the ms_since_start value which increases by about 100ms every line. For the parameters, only the 12 raw sensor values and the weight are selected. The other calculated values were useful for checking how the sensors exactly work, but would make the project too big if used in the model. After formatting the files, the tables only include the timestamp, the label and the sensor values.

Example sample

TIME (MS.)	LABEL (LABEL)	ACCX_RIGHT	ACCY_RIGHT	ACCZ_RIGHT
Sample 1 (labels: correct form)				
0	correct form	-2491.0	-457.0	2984.0
100	correct form	-2865.0	-236.0	3243.0
200	correct form	-2783.0	-389.0	3103.0
300	correct form	-2639.0	-336.0	2955.0
400	correct form	-2621.0	-304.0	3378.0
500	correct form	-2493.0	-249.0	2842.0

Figure 16: CSV Wizard

To create an Impulse, three parts are important:

Time Series Data:

The way time series data works is by creating windows out of continuous data and evaluating them separately. For the window size, the average length of each window was used, which is about 2.7 seconds. The window increase was chosen at 300ms, which means that after the first time window is completed, the model will wait 300ms because again evaluating the last 2.7 seconds. If a rep is shorter than the window size, the rest of the window will be filled with null values which won't affect the evaluation.

Processing Blocks:

There are many ways data can be processed to lead to a better result. For this case, spectral analysis was used as it is very fitting for continuous movement. The weight isn't integrated yet as it only has a small effect on the sensor values which totally suffice for testing, but it wouldn't be processed and instead used as raw data as it is a constant value.

Classification:

Classification gets inputs from a processing block and is the part that predicts the final result. In this case, a normal classifier was used as it can easily categorize movement patterns into different labels.

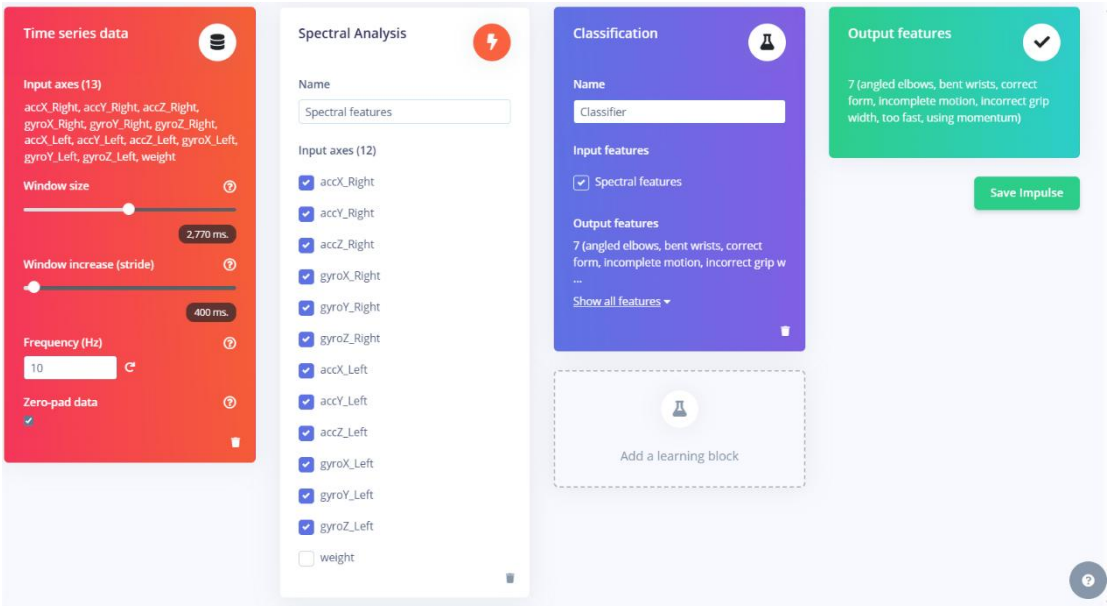


Figure 17: Form Detection Impulse

The spectral features are generated by processing the values using different parameters. Thankfully, Edge Impulse provides a reliable function to autotune these parameters based on the data set, which creates some filters which alter the data.

The values are then normalized to avoid differences in scaling having a big impact on the result. The generated features can be visualized to check if they separated correctly.

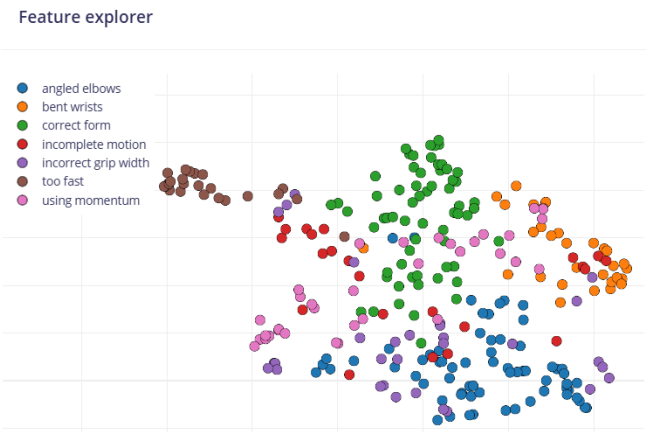
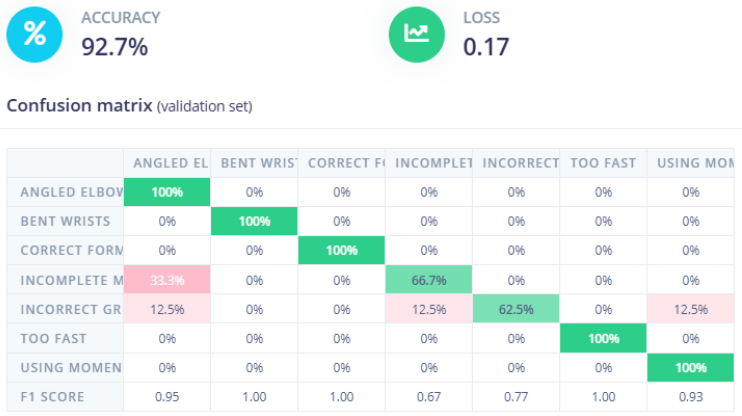


Figure 18: Form Detection Features

Here it is visible that while some labels (e.g. bent wrists) are in their own clusters, others often mix into other labels. This is still a very good result as the data set itself can't be perfect because there will always be slight human errors when performing a certain type of form.

When training the classifier with a train/test split of 80/20, 100 epochs and a learning rate of 0.0005, it results in an accuracy of 92.7% and a loss of 0.17. The loss is a penalty for a bad prediction and should be kept as low as possible. The model also has an average precision of 0.93 and an average recall of 0.92.



	ANGLED EL	BENT WRIS	CORRECT F	INCOMPLET	INCORRECT	TOO FAST	USING MOM
ANGLED ELBOV	100%	0%	0%	0%	0%	0%	0%
BENT WRISTS	0%	100%	0%	0%	0%	0%	0%
CORRECT FORM	0%	0%	100%	0%	0%	0%	0%
INCOMPLETE M	33.3%	0%	0%	66.7%	0%	0%	0%
INCORRECT GR	12.5%	0%	0%	12.5%	62.5%	0%	12.5%
TOO FAST	0%	0%	0%	0%	0%	100%	0%
USING MOMEN	0%	0%	0%	0%	0%	0%	100%
F1 SCORE	0.95	1.00	1.00	0.67	0.77	1.00	0.93

Figure 19: Form Detection Model

The model then gets tested on all testing samples and reaches an accuracy of 84.85%, with 10,4% of the predictions being incorrect and 4,75% being uncertain.



	ANGLED E	BENT WRIS	CORRECT I	INCOMPLE	INCORREC	TOO FAST	USING MO	UNCERTAI
ANGLED EL	75%	0%	0%	0%	6.3%	0%	12.5%	6.3%
BENT WRIS	0%	85.7%	0%	0%	0%	0%	0%	14.3%
CORRECT F	0%	0%	100%	0%	0%	0%	0%	0%
INCOMPLE	16.7%	0%	0%	66.7%	0%	0%	0%	16.7%
INCORREC	16.7%	0%	0%	16.7%	66.7%	0%	0%	0%
TOO FAST	0%	0%	0%	0%	0%	100%	0%	0%
USING MOI	0%	0%	14.3%	0%	0%	0%	85.7%	0%
F1 SCORE	0.80	0.92	0.97	0.73	0.73	1.00	0.80	

Figure 20: Form Detection Testing

This model can then be deployed as an Arduino library, using the Edge Impulse' EON Compiler which uses 45% less RAM and quantized int8 values to reduce the flash size to about 25kB.

4.3.2 Rep Detection

The data set for the rep detection is almost identical to the one used for form detection, only difference being the labels (running, end). The exact same CSV Wizard can be used to process the files.

The Impulse is also mostly the same, only the time series data is handled slightly differently. As there is no signal when a rep is finished, the average rep time can't be used as the window size because then it would not detect the end of a rep shorter than 2.7 seconds. Instead, it is set below the absolute minimum time of a rep at 600ms. The window increase is also slightly decreased to 300ms. As the end should always be detected, if both labels are present in a window it will always automatically be labeled as end.

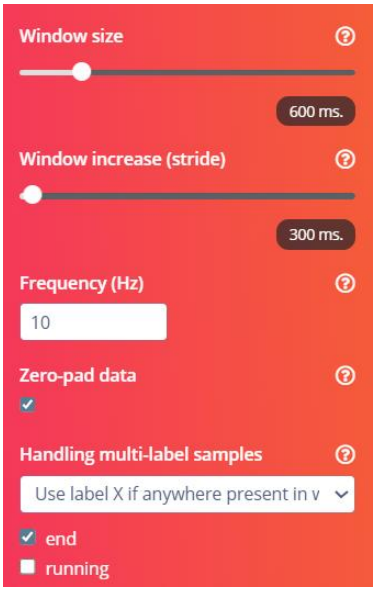


Figure 21: Rep Detection Time Series Data

After autotuning the spectral analysis parameters the features are again generated and the features normalized.

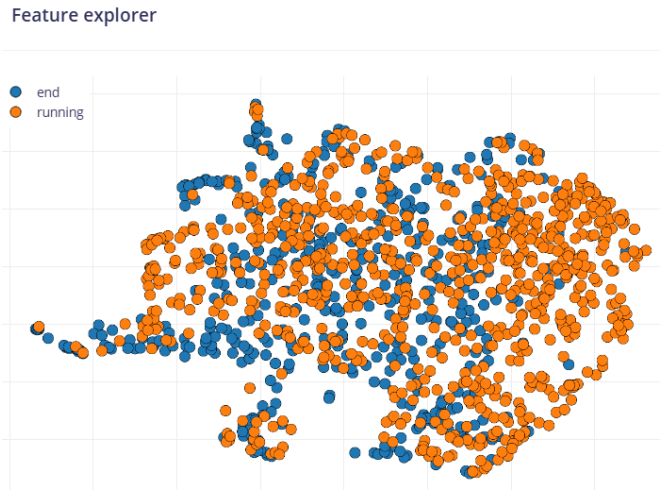


Figure 22: Rep Detection Features

The labels obviously overlap as they both just show different parts of the same movement. These 60/40 separations fluctuate slightly depending on the speed in which the exercise is performed. However, as each samples consists of multiple windows and each window is calculated on its own, in the context of this project the actual end of each repetition is calculated through the running/end ratio. Thus it doesn't have any effect if the evaluation has a slightly different offset every repetition.

The classifier was then trained with 100 epochs and a learning rate of 0.0005. The results were an accuracy of 79.1%, a loss of 0.47, and the average precision as well as recall being 0.79. The most important factor is a high recall on the end label as it should always be detected, and the value of 0.72 is, while not yet perfect, definitely usable.



	END	RUNNING
END	71.6%	28.4%
RUNNING	16.9%	83.1%
F1 SCORE	0.70	0.84

Figure 23: Rep Detection Model

Testing the set comes to an accuracy of 74.16%, with 17.56% being misclassified and 8.28% being uncertain. While this is almost 20% less accurate than the form detection, it still works well in the project as mentioned above. Additionally, some uncertainty is easily explainable through the overlap that is caused by different speeds.

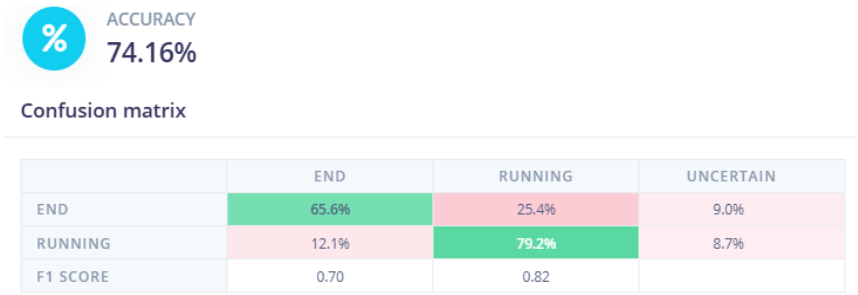


Figure 24: Rep Detection Testing

The model is then again deployed as an Arduino library using the EON-Compiler and quantized int8 values, resulting in a Flash size of about 16kB.

4.4 App Development

The implementation of the Android mobile application transforms the conceptual system design into functional software. Developed within Android Studio using the Flutter framework and the Dart programming language, the application needs to handle asynchronous hardware events, manage real-time user interfaces, and store data. This chapter is an overview of the software engineering approach, focusing on state management, the two-way Bluetooth Low Energy communication, and the active workout logic.

4.4.1 Early Prototype BLE Data Transfer Test

Before implementing the final, state-driven architecture and the AI-based evaluation logic, it was necessary to validate the foundational hardware-to-software communication. To achieve this, an initial Proof of Concept application was developed. This early prototype served as a milestone to verify that the Nicla Sense ME could successfully transmit continuous, high-frequency sensor data via Bluetooth Low Energy to a mobile device.

This Prototype of the app demonstrates the working data transfer as it receives all the raw sensor data from the gloves and outputs them directly on a live display.

4.4.1.1 Monolithic Architecture

Unlike the final application, which strictly separates UI and backend logic, the test application was built as a monolithic structure entirely contained within the `main.dart`.

The entire application state, including the BLE connection lifecycle (`_scanSubscription`, `_leftConnectionSubscription`) and the user interface, was managed within a single `StatefulWidget`. While this architecture is not scalable for multi-screen applications because navigating to a new screen would destroy the widget and sever the Bluetooth connection it was highly efficient for rapidly prototyping a single-screen data dashboard.

4.4.1.2 Raw Data Parsing

In this early iteration, the microcontroller did not send processed AI labels (like "correct form"), but rather the raw, unfiltered data received by the microcontroller. The Nicla Sense ME transmitted an array consisting of six specific data points: three axes for the Accelerometer (X, Y, Z) and three axes for the Gyroscope (X, Y, Z).

To conserve bandwidth, the microcontroller packaged these six floating-point numbers into a tight binary array of 24 bytes (4 bytes per 32-bit float). A critical challenge in the app implementation was unpacking this byte array back into readable decimal numbers. Since the hardware transmits data in a Little Endian format (where the least significant byte is stored first), the application used Dart's `ByteData` class to accurately parse the incoming stream.

```
// Snippet from the early prototype: Parsing binary sensor data
List<double> _bytesToFloatList(Uint8List bytes) {
  // Ensure the byte array is a multiple of 4 (32-bit float = 4 bytes)
  if (bytes.length % 4 != 0) return [];

  final bd = ByteData.sublistView(bytes);
  final list = <double>[];

  // Iterate through the array in steps of 4 bytes
  for (int i = 0; i < bytes.length; i += 4) {
    // Extract the 32-bit float using Little Endian byte order
    list.add(bd.getFloat32(i, Endian.little));
  }
  return list;
}
```

4.4.1.3 UI Rendering and Limitations

After successfully parsing the 24-byte payload into a Dart `List<double>` containing six values, the UI updated dynamically. The screen featured a color-coded status card for the BLE connection and a custom `_buildSensorGroup` widget that mapped the incoming array to a grid, displaying the real-time X, Y, and Z values for both acceleration and rotation.

Lessons Learned: While the prototype successfully proved that the BLE pipeline was functional and robust enough for high-frequency data streams, it also highlighted the limitations of passing raw data. Transmitting 24 bytes continuously consumed significant battery power on the microcontroller and required the smartphone to render UI updates at an aggressive rate.

These findings directly influenced the architectural decisions for the final application. By shifting the heavy mathematical processing to the Edge AI on the Nicla Sense ME itself, the final application only needed to receive a single 1-byte label upon the completion of a repetition, drastically reducing latency and power consumption.

4.4.2 Architecture and State Management Approach

The app's source code is divided into specific files, each responsible for a distinct layer of the system architecture. The core logic is distributed across four main Dart files:

- **`models.dart` (Data Layer):** This file defines the domain models of the application. It contains the data structures `Exercise`, `WorkoutSet`, and `WorkoutSession`. These classes contain the properties of a workout (e.g., target repetitions, weight, AI feedback) and include the necessary logic to serialize and deserialize the data into JSON format for local storage.

- **`ble_service.dart` (Hardware Integration):** This is the core communication module. It is completely separated from the User Interface and handles the entire Bluetooth lifecycle: requesting device permissions, scanning for the Nicla Sense ME microcontroller, managing the connection state, and translating incoming byte streams into readable fitness data.

- **`active_workout_screen.dart` (Dynamic Presentation during Workout):** This file hosts the interface that the user sees during an ongoing exercise. It mostly depends on real-time data and contains the logic for automatically transitioning between sets based on the incoming sensor data.

- **`main.dart` (Application Entry and Main Presentation):** This file serves as the entry point of the application. It initializes the global state provider and contains the routing logic. Additionally, it hosts the static and configuration-based screens, including the Start Screen, the Main Menu, the Workout Setup Screen, the Summary Screen, and the Workout History.

4.4.3 Ensuring Continuous BLE Connectivity via Global State

In a standard Flutter application, localized state is often managed using the `setState()` method. However, for an application communicating with external hardware, this approach is insufficient. If the BLE connection logic were tied to a specific screen, navigating to a different menu would destroy the connection and the workout progress.

To solve this, the application architecture implements the **Provider** state management pattern. The `BleService` class extends Dart's `ChangeNotifier`. It is incorporated at the very root of the application using a `MultiProvider` widget.

```
// Snippet from main.dart: Initializing the global state
void main() {
  runApp(
    MultiProvider(
      providers: [
        // The BleService is initialized once and kept alive globally
        ChangeNotifierProvider(create: (_) => BleService()..init())
      ],
      child: const FormerApp(),
    ),
  );
}
```

This architectural decision ensures that the `BleService` works at all times. The BLE connection remains stable, and the sensor data is retained continuously in the background, regardless of how the user navigates through the visual screens.

4.4.4 Two-Way BLE Communication Module

The communication relies on the `flutter_reactive_ble` library to establish a low-latency connection with the Nicla Sense ME. The `BleService` manages a sequence of asynchronous operations to ensure a stable data flow.

Upon connection, the service first requests a Maximum Transmission Unit (MTU) of 32 bytes (`_requestMtu`). This ensures that the data packets containing the setup parameters and feedback labels are not fragmented by the operating system, which would increase processing overhead.

1. Transmitting Workout Parameters (App -> Glove)

Before an exercise begins, the glove's internal AI needs context to classify the movements correctly. The app actively writes the workout configuration to a specific "RX" (Receive) Characteristic on the glove.

```
// Snippet from ble_service.dart: Sending setup parameters to the hardware
Future<void> sendWorkoutSetup(int startCommand, int exerciseId, int targetReps, double weightKg) async {
  if (!isConnected || connectedDeviceId == null) return;

  final characteristic = QualifiedCharacteristic(
    serviceId: serviceUuid,
    characteristicId: rxCharUuid, // Target characteristic for commands
    deviceId: connectedDeviceId!,
  );

  // Transmitting an array of integers representing the workout blueprint
  await _ble.writeCharacteristicWithResponse(
    characteristic,
    value: [startCommand, exerciseId, targetReps, weightKg.round()],
  );
}
```

2. Receiving Real-Time AI Feedback (Glove -> App)

Once the workout is active, the app processes the AI's classification results. Instead of inefficiently polling the hardware, the application subscribes to a Notification stream on the `labelCharUuid` characteristic.

Whenever the Edge AI classifies a movement, it transmits a single byte representing a label index. The `BleService` listens to this stream, increments the internal repetition counter, and translates the numeric index into a readable, qualitative feedback string (e.g. mapping index 1 to “bent wrists” and index 2 to “correct form”).

```
// Snippet from ble_service.dart: Subscribing to AI notifications
_dataSub = _ble.subscribeToCharacteristic(characteristic).listen((data) {
  if (data.isNotEmpty) {
    int labelIndex = data[0]; // Extract the AI classification integer
    currentReps++; // Increment global rep counter

    // Translate the integer into a qualitative feedback string
    String feedback = _getFeedbackForLabel(labelIndex);
    currentFeedback = feedback;
    currentSetFeedbacks.add(feedback); // Append to the current set's history

    notifyListeners(); // Broadcast the state change to the UI
  }
});
```

The `notifyListeners()` call is the critical trigger in this architecture. It alerts the Flutter framework that new data is available, prompting a visual update.

4.4.5 Real-Time UI activity (Active Workout Screen)

The `ActiveWorkoutScreen` must present incoming data to the athlete instantly without lagging. To achieve this while sticking to the Minimal Touch Interaction principle, the UI is wrapped in a `Consumer<BleService>` widget.

The `Consumer` widget listens to the broadcasts sent by `notifyListeners()`. When a new repetition is registered, only the widgets inside the `Consumer` (such as the `CircularProgressIndicator` and the text displaying the repetitions) are rebuilt.

Furthermore, the application logic continuously evaluates the current repetitions against the predefined target. To avoid frame drops or exceptions caused by state modifications during the UI rendering phase, the transition to the next set is scheduled using `addPostFrameCallback`.

```
// Snippet from active_workout_screen.dart: Automated set progression
return Consumer<BleService>(
  builder: (context, bleService, child) {
    // Check if the target repetitions are reached
    if (bleService.currentReps >= currentSet.targetReps && !isFinished) {
      // Safely schedule the transition to the next set after the frame renders
      WidgetsBinding.instance.addPostFrameCallback((_) => _nextSet());
    }

    // Build the visual progress indicator
    return CircularProgressIndicator(
      value: (bleService.currentReps / currentSet.targetReps).clamp(0.0, 1.0),
      // ... styling properties ...
    );
  }
);
```

This mechanism ensures that once a set is completed, the app autonomously saves the current performance, resets the hardware counters, and proceeds to the next exercise phase without requiring physical input from the user.

4.4.6 Data Serialization and Local Storage

After a training session ends, the gathered data must be preserved to allow users to review their progression and AI evaluations over time. For this prototype, persistent local storage was implemented using the `shared_preferences` package, which accesses the native key-value storage of the Android operating system.

Because `shared_preferences` can only store basic data types (like strings, integers, or lists of strings), the complex, nested data structures of the workout must be serialized.

1. **Serialization:** The `WorkoutSession` object (which contains a list of `WorkoutSet` objects, which in turn contain `Exercise` data and the AI feedback strings) is converted into a standard JSON string using the custom `toJson()` methods defined in `models.dart`.

2. **Storage:** This JSON string is appended to a string list and saved to the device memory.
3. **Deserialization:** When the user navigates to the `HistoryScreen`, these JSON strings are retrieved, decoded via `jsonDecode()`, and mapped back into functional Dart objects using the `fromJson()` factory constructors.

5 User Case

To use the Former Glove, both the gloves and the Former app installed on your mobile device are needed. Firstly, the battery connectors have to be plugged in in order to supply the microcontrollers. After that, the app is started and automatically connects to the gloves over Bluetooth.

To start training, a new training session can be created in the app. One selects the exercises they want to perform, then they input the weight, the amount of sets and the amount of reps per set. Each set can be started whenever the user is ready. Once a set has begun, the user simply trains until the set is completed. Each rep is analyzed and the form is evaluated to give live feedback in the app and after completion of the set number of repetitions, the app automatically jumps to the next set of reps. Once a training session is completed, there is a summary of which mistakes the user has made. The results are saved and the user can review their performance in previous training sessions at all times in the “Workout History” section.

To turn off the gloves, the connectors have to be unplugged again. To charge the integrated batteries, the Nicla Sense ME has to be connected with a micro-USB cable.

6 Evaluation

To test if the final product works and evaluate how well it works, all three components were powered simultaneously to test the reaction. After a few seconds, the left glove found the right glove and the app immediately connected. To test the AI model itself, seven sets of 10 bench presses each were selected, one set to perform each label that has been trained.

Three major factors were important: The latency in which the movement was evaluated, if repetitions were recognized and the actual real-time accuracy of the model. The latency turned out not to be a problem, as Edge Impulse already provided the evaluation latency of 3ms and receiving the label in the app takes around 50ms. Data only updates every 100ms, thus there are no discrepancies. The rep detection model evaluates the end of a repetition correctly, however, it only works if the user starts immediately after beginning a set as no way of detecting when the user actually begins exercising has been implemented yet.

After performing all 7 sets, an accuracy of 95% was reached, which is similar to the classifier accuracy in Edge Impulse. The reason it isn't lower as was the case during model testing is that there is no uncertainty, the highest scoring label is always chosen even if it has a probability below 70%. With a precision of 0.94 and a recall of 0.92 it is also very similar to the classifier. It also has to be noted that the model is currently only trained on one specific barbell, and more varied setups will have to be recorded for a product that functions universally.

	ANGLED ELBOWS	BENT WRISTS	CORRECT FORM	INCOMPLETE MOTION	INCORRECT GRIP	TOO FAST	USING MOMENTUM
ANGLED ELBOWS	100%	0%	0%	0%	0%	0%	0%
BENT WRISTS	0%	100%	0%	0%	0%	0%	0%
CORRECT FORM	0%	0%	100%	0%	0%	0%	0%
INCOMPLETE MOTION	10%	0%	0%	90%	0%	0%	0%
INCORRECT GRIP WIDTH	10%	0%	0%	20%	70%	0%	0%
TOO FAST	10%	0%	0%	0%	0%	90%	0%
USING MOMENTUM	0%	0%	0%	0%	0%	0%	100%

Figure 25: Evaluation Confusion Matrix

7 Project Management

7.1 Planning

Communication between App and Gloves

Date: 29th October 2025

A test code was written which confirms the gloves can exchange sensor data and can send it to the app.

System fully integrated into Gloves

Date: 5th November 2025

The housing of both the microcontroller and battery including the mounting rail were modelled, printed and sewn onto the gloves. The microcontrollers and batteries were placed inside the housings and the connectors were soldered to the corresponding pins.

Working Data Transfer between App and Gloves

Date: 5th December 2025

The App had a working fundamental hardware-to-software bridge, received raw sensor data from the Nicla Sense ME and displayed the data on the screen with live dynamic updates.

Complete Data Set

Date: 28th January 2026

240 bench press reps were recorded in a training session and uploaded to Edge Impulse as csv files to create a data set for bench presses.

AI trained

Date: 4th February 2026

The AI for form recognition was fully trained and adjusted to come to a final accuracy of just under 93%.

AI integration onto microcontrollers

Date: 25th February 2026

The Edge Impulse libraries were integrated into the microcontrollers to run the final code. The form recognition runs on the left glove while the rep recognition runs on the right glove.

App UI Completely finished

25th February 2026

This milestone marks the finalization of the application's frontend architecture. All planned user interfaces, from the initial workout setup to the detailed performance history, have been successfully implemented, translating the planned user experience principles into a fully navigable, polished application.

7.2 Reflection

All in all, the project included a fair share of problems of which solving them was essential to keep the approach working, while in the end, everything ended up working almost exactly as was planned at the beginning. The biggest change was the overall structure of communication, as the Nicla Sense ME turned out to only allow one client per server. Thus, it was made sure the left glove functions as both a server and a client at the same time. On the software side, establishing a continuous connection to this left glove presented its own architectural challenges. Because standard UI navigation in the Flutter app would normally destroy the active Bluetooth stream, the BLE communication had to be strictly decoupled from the visual screens using a globally persistent state management system (Provider). Another slight change was distributing the AI models over both microcontrollers, as Edge Impulse always uses the same defined values and there is no feasible way of changing over 3000 files. This was actually helpful as the flash memory of both microcontrollers could be used more evenly. For the evaluation, it seemed like a much simpler version of the form detection AI would have to be used in order to fit on the 512 kB flash memory of the Nicla Sense ME. The reason this problem occurred is because in order to use the sensors and BLE function of said controller, more than half of the available flash memory had to be used.

This wasn’t documented on the datasheet of the Nicla Sense ME, which seems like a dealbreaker because these are both presented as basic functions and it is even advertised as being suitable for running AI models. On a positive note, it was still possible to run the whole bench press model as Edge Impulse used too many layers for the complexity of the model. Despite this, the Nicla Sense ME would still limit the expansion of the exercise library.

Parts of the project that worked almost flawlessly include the communication of sensor values between gloves, the recording of movement data and the training of the AI model. Similarly, translating this incoming data into a responsive User Interface worked exceptionally well. The implementation of the Minimal Touch Interaction concept allowed the mobile app to automatically increment repetitions and transition between sets based purely on the hardware's event triggers, ensuring a hands-free experience for the user. The 3D-modelled housing took a few attempts to get right, but the final result satisfied the expectations and worked perfectly as a prototype as the housings could easily be slid off the mounting rail. Generating the features of the form detection actually worked much better than expected. It seemed that recording only hand motion to detect e.g. elbows being angled wrong, however the labels were seamlessly separated. Finally, taking these labels and presenting them as readable, qualitative feedback on the app's summary screen brought the whole concept together. Serializing these workout sessions into JSON for local storage provided a reliable history of the user's performance, rounding off the hardware prototype into a complete, user-centric product.

7.3 Timesheet

7.3.1 Daniel Jones

Calendar Week	Task	Hours (Free Time)	Hours (School)
Before Week 38	Motion data exchange between Nicla Sense ME microcontrollers	36.0	7.5
38	ABA portal updated, cost plan	4.0	7.5
39	Switched to VS Code, work submitted on ABA portal	6.0	7.5
40	3D model	8.0	7.5
41	3D model, planning battery connection	10.0	7.5
42	3D model	4.0	7.5
43	3D model, µC-app interface, data transfer to Edge Impulse	6.0	7.5
44	µC-app interface	10.0	0.0
45	Nicla Sense ME + battery mounted with enclosures on the gloves	6.0	7.5
46	Battery code	6.0	7.5
47	Battery code	6.0	7.5

Calendar Week	Task	Hours (Free Time)	Hours (School)
48	Youth Innovates (Jugend Innovativ) submission	6.0	7.5
49	Preparation for Open Day, sensor values → CSV file	10.0	7.5
50	Code for repetition detection	6.0	0.0
51	Data collection	6.0	7.5
1	Improving repetition detection (considering weight)	3.0	0.0
2	Data collection	6.0	7.5
3	Data collection	6.0	7.5
4	Data collection	6.0	7.5
5	Data collection, Edge Impulse	8.0	7.5
6	Edge Impulse, integrating AI on µC	6.0	7.5
7	AI repetition detection, AI on µC, thesis writing, app transfer	10.0	7.5
9	Milestone: AI integrated on µC, writing diploma thesis	6.0	7.5
10	Writing diploma thesis	6.0	7.5
11	Writing diploma thesis	6.0	7.5

Calendar Week	Task	Hours (Free Time)	Hours (School)
12	Writing diploma thesis	6.0	7.5
13	Writing diploma thesis	6.0	7.5
---	---	---	---
Total		**205.0**	**187.5**

…….……..... ……..…….

Ort und Datum Vorname Nachname

7.3.2 Kevin Silajdzic

Calendar Week	Task	Hours (Free Time)	Hours (School)
Before Week 38	Android Studio setup	31.0	7.5
38	ABA portal updated, Android emulator setup	4.0	7.5
39	First test app, work submitted to ABA portal	6.0	7.5
40	App development	6.0	7.5
41	Planning battery connection	10.0	7.5
42	App development	6.0	7.5
43	µC-App interface	6.0	7.5
44	µC-App interface	7.0	0.0
45	Nicla Sense ME + battery mounted with enclosures on the gloves	1.0	7.5
46	Newspaper article, poster, app programming	6.0	0.0
47	App programming	6.0	7.5
48	Youth Innovates (Jugend Innovativ) submission, app programming	6.0	7.5
49	Preparation for Open Day	10.0	7.5
50	Expanding app functions	6.0	0.0

Calendar Week	Task	Hours (Free Time)	Hours (School)
51	Expanding app functions	6.0	7.5
1	Expanding app functions	8.0	7.5
2	Expanding app functions	3.0	7.5
4	Improving app UI, data collection	6.0	7.5
5	Data collection, writing diploma thesis	8.0	7.5
6	Writing diploma thesis	6.0	0.0
7	Data transfer to the app	6.0	7.5
8	App bug fixes	5.0	0.0
9	Writing diploma thesis	6.0	0.0
10	Writing diploma thesis, training history feature	6.0	7.5
11	Writing diploma thesis, training history feature	6.0	7.5
12	Writing diploma thesis, app UI improvements	6.0	7.5
13	Writing diploma thesis	6.0	7.5
---	---	---	---
Total		**189.0**	**157.5**

…….……..... …….……

Ort und Datum Vorname Nachname

8 Future Work

In the future, the usage of a different microcontroller will be crucial to fix the limited flash memory of the Nicla Sense ME. A perfect substitution would be the XIAO nRF52840 Sense by Seeed Studio. It is even smaller than the Nicla Sense ME, has four times the flash memory and uses C++ so the code can stay almost exactly the same. The only part that would be changed are the libraries used for the sensors as it doesn't use Bosch' accelerometer and gyroscope.

(Source: https://wiki.seeedstudio.com, 03-26-2026)

The integration of the hardware into the gloves could also use an update to make it more professional. The microcontrollers and battery should be directly sewn into a thicker glove to make exercising with it even more comfortable. This would require a new way to charge the battery, which could be done e.g. by fixing the microcontroller's socket to a small opening in the glove.

Both the microcontroller and the app should be updated to include much more exercises such as deadlifts, lat raises and overhead presses. This would require a lot of data sets and a way to handle multiple exercises with one AI model. Additionally, users should be able to agree that their own recorded movement data is used to improve the AI based on different parameters which you can choose to provide in the app (e.g. height, weight, age etc.).

The Former App should include a community where one can create a profile and share their current performances and training sessions with other users. This can be expanded into an XP system with different rewards to motivate the user. Consistent further development of the app is a key factor in making the product appealing for customers.

9 Related Work

Among fitness wearables, the Former Glove is unique in its ability to analyse the user's form and give feedback. However, there are still some products which perform similar functions.

Shape.Care (Smart Gym Gloves)

- **Technical Approach:** Shape.Care uses professional training gloves integrated with multiple sensors (IMUs and pressure sensors) connected via conductive threads to a central processing hub on the back of the hand. It tracks movement parameters, tempo, and breaks, providing real-time feedback through a mobile app.
- **Differentiation:** While Shape.Care offers a similar hardware form factor, the Former Glove distinguishes itself through its specific AI-driven form classification. Instead of just measuring general parameters, the Former Glove focuses on identifying qualitative errors. Additionally, the Former Glove emphasizes being a more affordable and privacy-compliant solution by processing motion data without the need for complex, high-cost sensor arrays or external tracking.

(Source: https://www.crunchbase.com/organization/shape-care, 03-25-2026)

Peloton Guide / AI Camera Systems

- **Technical Approach:** These systems use Computer Vision and high-resolution cameras to track skeletal keypoints of the user's body. AI models analyse these video streams to count repetitions and provide visual feedback on the user's posture and range of motion.
- **Differentiation:** The Former Glove is mobile and hardware-integrated, meaning it does not require a stationary camera setup or a TV, making it usable in any gym environment. Unlike camera systems, it solves the privacy concern, as no video data is ever recorded or processed. Furthermore, the glove-based sensors can capture fine-tuned hand and wrist stability data that cameras might miss due to occlusion (e.g. if a weight plate or body part blocks the camera's view of the hand).

(Source: https://www.onepeloton.com, 03-25-2026)

10 Conclusion

The Former Glove project was developed to fill a critical gap in strength training: the lack of immediate, qualitative feedback. While traditional solutions rely on expensive personal trainers or stationary camera setups, our system offers a mobile alternative.

By combining precise motion sensors directly in the glove with intelligent data processing, we realized a fully functional prototype. The system is capable of distinguishing clean repetitions from common errors, such as using momentum. By splitting tasks by counting repetitions on the right glove and analysing form on the left glove, the system remains highly responsive and efficient via Bluetooth.

The accompanying Former App acts as a digital coach. During development, we prioritized easy control to ensure the athlete isn't distracted during a set. Live visual feedback allows users to correct their technique instantly, reducing the risk of injury and improving overall training results.

In summary, the Former Glove demonstrates that wearable sensor technology can make professional coaching accessible to everyone. The system provides a solid, scalable foundation for adding more exercises in the future, making a practical contribution to safe and effective training.

Acknowledgements

We would like to express our gratitude to everyone who supported us during the development of the Former Glove project.

First and foremost, we would like to thank our supervisor, Dipl.-Ing. Mario Preishuber, for his constant guidance, technical expertise, and valuable feedback throughout this thesis. His support was essential in overcoming the various challenges we faced.

We would also like to thank the students in our lab group for their practical help and the collaborative atmosphere. Their input and assistance during the testing and development phases were greatly appreciated.

Finally, we are deeply grateful to our families for their unconditional support and encouragement during our time at school and especially during the final stages of this project.

List of Figures

Figure 1: Process 11

Figure 2: System Architecture 12

Figure 3: Workout History 14

Figure 4: Workout Setup 15

Figure 5: Running Set 16

Figure 6: Summary Screen 17

Figure 7: Arduino Nicla Sense ME Mechanical Details 18

Figure 8: Arduino Nicla Sense ME 18

Figure 9: Nicla Sense ME Housing v1 34

Figure 10: Battery Housing v1 34

Figure 11: Mounting Rail 34

Figure 12: Battery Housing v2 35

Figure 13: Nicla Sense ME Housing v2 35

Figure 14: Housing sliced in Bambu Studio 35

Figure 15: Prototype 36

Figure 16: CSV Wizard 37

Figure 17: Form Detection Impulse 38

Figure 18: Form Detection Features 38

Figure 19: Form Detection Model 39

Figure 20: Form Detection Testing 39

Figure 21: Rep Detection Time Series Data 40

Figure 22: Rep Detection Features.....41

Figure 23: Rep Detection Model 41

Figure 24: Rep Detection Testing 42

Figure 25: Evaluation Confusion Matrix 52

DANIEL JONES

5121 Ostermiething, Austria
+43 0677 625 79405
danedjo18@gmail.com
4th September 2006, Poole (UK)



EDUCATION

HTL Braunau
Electronics & Computer Science - Coding & A.I.
2021 - Present
Expected to graduate in 2026

Rottmayr-Gymnasium Laufen
2017 - 2021

VS Ostermiething
2013 - 2017

PROJECTS

BTEQ Box
School Project
Created a Bluetooth music box with an in-built equalizer using a self-made circuit and 3D-printed housing.

Former Glove
Diploma Thesis
Smart AI-based training gloves with integrated microcontrollers and sensors which analyse the user's form in an app while exercising.

EXPERIENCE

Mandatory Internship
Senswork Burghausen
July 2025
- Designed and assembled AI camera interface in KiCad
- Labeled image samples for an external client
- Performed sensor-based defect analysis on components
- Programmed robot arm for multi-label scanning

Mandatory Internship in Warehouse Operations
B&R Industrial Automation, Eggelsberg
August 2023
- Labeling incoming shipments
- Scanning goods into the inventory system

SKILLS

Software Engineering
- Java
- C++
- Python
- Embedded Coding

Hardware & Electronics
- KiCad
- Circuit Design & Assembly
- Sensor Systems

Soft Skills:
- Project Coordination & Planning
- Teamwork
- Ambitious & goal-oriented

Other:
- German & English as native languages
- Basic knowledge in French & Dutch
- Category B driver's license

HOBBIES

Drums
- Spielmannszug Braunau since 2016
- Marktmusikkapelle Ostermiething since 2017

Sports
- Tennis
- Football
- Basketball

KEVIN SILAJDZIC

5211 Friedburg, Austria
+43 0644 4935509
kevinsil2707@gmail.com
27th July 2007, Braunau (AT)



EDUCATION

HTL Braunau
Electronics & Computer Science - Coding & A.I.
2021 - Present
Expected to graduate in 2026

Neue Mittelschule Friedburg
2017 - 2021

Volksschule Friedburg
2013 - 2017

PROJECTS

BTEQ Box
School Project
Created a Bluetooth music box with an in-built equalizer using a self-made circuit and 3D-printed housing.

Former Glove
Diploma Thesis
Smart AI-based training gloves with integrated microcontrollers and sensors which analyse the user's form in an app while exercising.

EXPERIENCE

Mandatory Internship
Werbeagentur Nex
July - August 2024

Mandatory Internship
Werbeagentur Nex
July - August 2023

SKILLS

Software Engineering
- C
- C++
- Java
- Python
- Dart

Electronics
Soft Skills:
- Creative
- Analytic way of thinking

Other:
- German (Native language)
- English (C1)
- Category B driver's license

HOBBIES

Music Production
Going for walks
Video Games